Sovereign SQL Engine

Enterprise-Grade Text-to-SQL Platform with Real-Time Streaming, Self-Correction & Cloud-Native Observability

Laksh Mendpara (B23CS1037) Sahil Preet Singh (B23CS1061)

April 21, 2026

Abstract

Natural language interfaces for databases allow non-technical users to query structured data without writing SQL. However, existing solutions suffer from schema hallucination, high inference latency, prohibitive GPU costs, and a lack of real-time feedback. The **Sovereign SQL Engine** addresses these limitations through a multi-tier, event-driven architecture that combines semantic vector retrieval (Pinecone), graph-based relationship expansion (Neo4j), and serverless GPU inference (Modal & RunPod). The system introduces a novel self-correction loop where execution errors are automatically diagnosed and repaired by an advanced reasoning model. Aggressive infrastructure optimizations—including AWQ 4-bit quantization, container memory snapshots, FlashBoot peer-to-peer caching, and immutable SSE stream caching—reduce cold-start times to under 2 seconds and idle costs to $0.00/hour. This report provides an exhaustive technical account of the architecture, pipeline logic, optimization strategies, observability stack, and empirical performance results.

Project Links:

- **Live Deployment:** https://app.lakshmendpara.tech/soverign-sql/
- **Demo Video:** https://drive.google.com/file/d/1cyZJrekdaJdWI3XmbbMfacIqB4hBzNM6/view?usp=sharing
- **GitHub Repository:** https://github.com/Laksh-Mendpara/sovereign-sql-engine
- **Experiments Done:** https://docs.google.com/spreadsheets/d/1YakUTSlEHUiUjMrZTdnFF7AyFZU8/edit?usp=sharing

Contents

1 Introduction

In modern data-driven enterprises, querying analytical warehouses traditionally requires explicit SQL proficiency. Data analysts, product managers, and engineers face constant friction when exploring proprietary metrics across schemas that span hundreds of interrelated tables with ambiguous naming conventions.

Recent advances in Large Language Models (LLMs) have demonstrated extraordinary code-generation capabilities. However, applying zero-shot models to private enterprise databases introduces three critical challenges:

1. **Data Privacy & Sovereignty.** Transmitting proprietary schema definitions, row-level statistics, or live query results to third-party APIs (e.g., OpenAI, Anthropic) violates compliance frameworks such as GDPR and HIPAA.

2. **Schema Hallucination.** Without explicit knowledge of the target database design, LLMs routinely generate references to non-existent tables, invent invalid JOIN paths, or produce syntactically correct but semantically incorrect queries.

3. **Latency & Idle Cost.** Maintaining permanently provisioned GPU instances for unpredictable Text-to-SQL traffic patterns leads to unmanageable operational expenditures that scale poorly.

The Sovereign SQL Engine resolves each of these bottlenecks by deploying a distributed ensemble of specialized micro-models orchestrated through a lightweight, fully asynchronous backend. Rather than relying on a single monolithic model, the system routes tasks to purpose-built inference nodes—safety auditing on NVIDIA T4s, classification on L4s, and deep SQL reasoning on A100/H100 clusters—each independently scalable to zero.

2 System Architecture

The platform is designed as a three-layer distributed system that eliminates blocking I/O in favor of a fully event-driven, non-blocking asynchronous pattern managed by Python's `asyncio`.

2.1 Layer 1: Presentation (React Frontend)

The client interface is built on React 18 with Vite. It establishes a persistent **Server-Sent Events (SSE)** connection to the backend. Unlike traditional REST polling, the UI receives highly granular, incremental payloads—each pipeline stage (safety check, classification, vector retrieval, graph expansion, SQL generation) is rendered as a live "Chain of Thought" card the instant it resolves. This micro-latency feedback loop ensures the user perceives immediate responsiveness even while expensive GPU inference is still in progress.

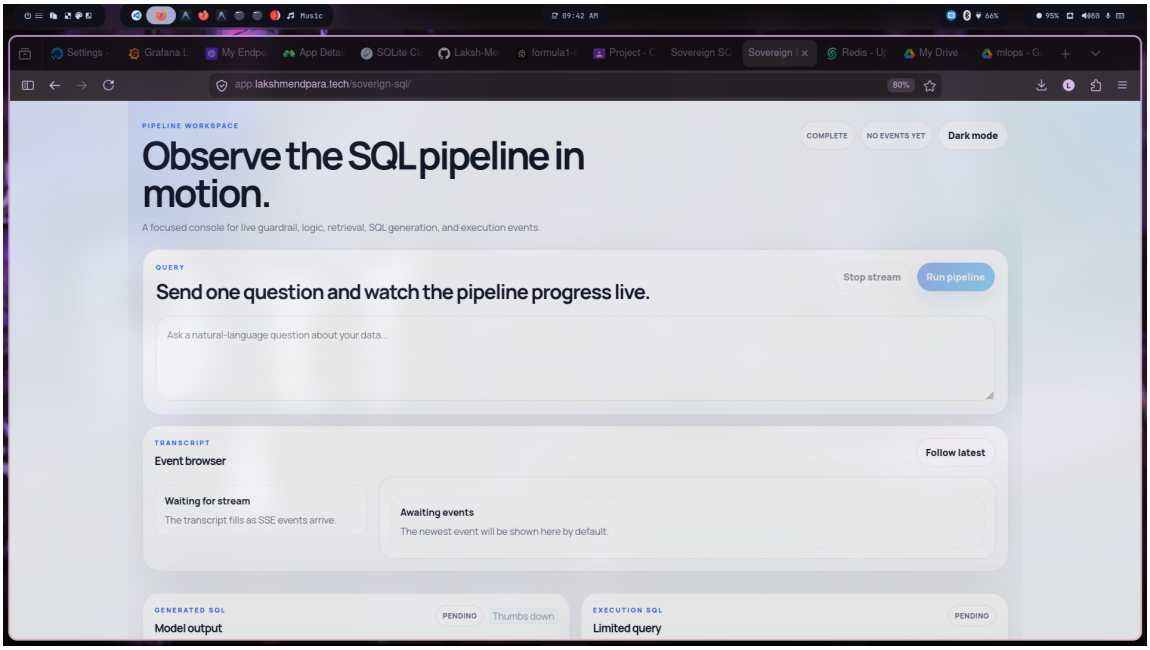

Figure 1: The Sovereign SQL Engine user interface showing the real-time Chain of Thought rendering during query processing.

2.2 Layer 2: Orchestration (FastAPI Backend)

The backend operates as the singular routing brain of the platform. Implemented in FastAPI running behind the Uvicorn ASGI server, it excels at network multiplexing. The custom `SSEPipelineExecutor` dispatches independent tasks concurrently using `asyncio.gather`—for example, the Llama Guard safety check, Phi-4 classification, and Pinecone vector retrieval all execute in parallel. Each resolved stage pushes its result into an `asyncio.Queue`, which the SSE serializer drains into the HTTP response stream immediately.

Key backend responsibilities include:

- SSE pipeline orchestration with per-stage latency tracking.
- Connection pool management for SQLite Cloud, Pinecone, and Neo4j.
- In-memory LRU caching of complete SSE event sequences.
- Prometheus metrics exposition and external RunPod health polling.
- Structured JSON logging with distributed trace ID propagation.

2.3 Layer 3: Inference & Storage

The data layer is heavily federated across purpose-built services:

- **Inference (Modal):** Hosts Phi-4-mini (NVIDIA L4, 24 GB) and Llama Guard 3 (NVIDIA T4, 16 GB) via vLLM with AWQ 4-bit quantization.
- **Inference (RunPod):** Hosts Arctic and Qwen3 on elastic A100/H100 serverless workers for SQL generation and self-correction.
- **Vector Store (Pinecone):** 1024-dimensional dense embeddings for semantic table and column retrieval.

- **Graph Store (Neo4j):** Property graph encoding all foreign-key relationships for JOIN path discovery.
- **Metadata (SQLite Cloud):** Distributed edge database storing table DDLs, column descriptions, and audit logs.

3 The Text-to-SQL Pipeline

The pipeline executes across three carefully sequenced tiers, each emitting real-time SSE events.

3.1 Tier 1: Concurrent Validation & Classification

Upon receiving a query, the orchestrator simultaneously dispatches three independent tasks:

(A) Safety Audit (Llama Guard 3). The raw user text is forwarded to a Modal-hosted Llama Guard 3 instance. This model evaluates the input against a strict safety taxonomy, detecting prompt-injection attacks, malicious DROP/DELETE derivations, and jailbreak attempts. If the input is flagged as unsafe, the pipeline terminates immediately with a `guard` SSE event containing `allowed: false`. The user interface renders a clear rejection notice.

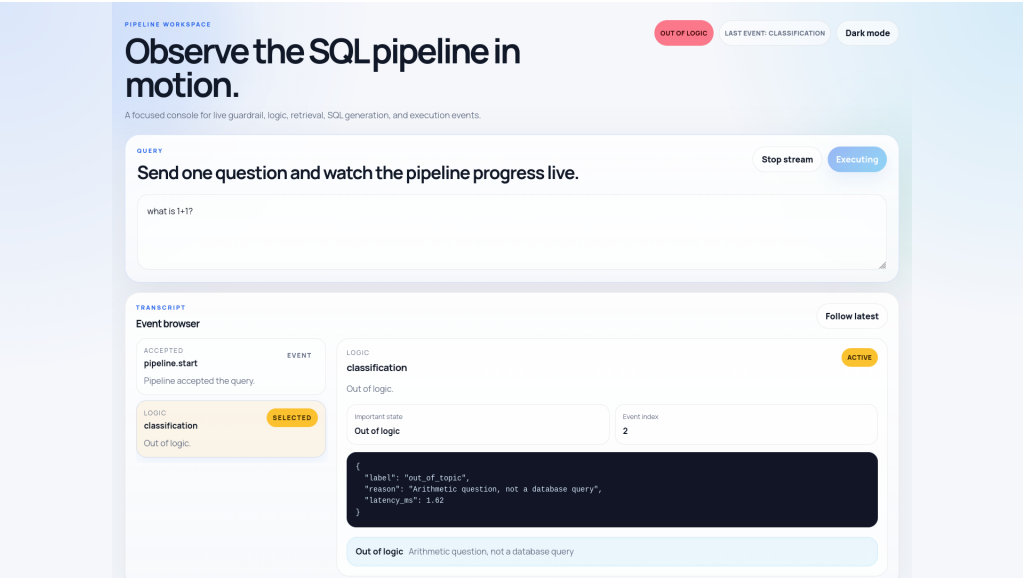

Figure 2: Frontend rendering of a blocked query after the Llama Guard 3 safety audit flags the input as unsafe.

(B) Difficulty Classification (Phi-4-mini). In parallel, the same query is sent to a Phi-4-mini instance. This model categorizes the question as *easy* (single-table filters), *difficult* (multi-table JOINs, subqueries, aggregations), or *out_of_topic*. This classification drives downstream routing decisions—difficult queries trigger a secondary enhancement pass through the advanced reasoning model.

(C) Vector Retrieval (Pinecone). Also in parallel, the query is embedded into a 1024-dimensional dense vector using Pinecone's `llama-text-embed-v2` model. The system performs two concurrent semantic searches: one against the *column* namespace and one against the *table* namespace. Results are post-processed through Pinecone's `pinecone-rerank-v0` cross-encoder to produce a ranked list of the most relevant schema elements.

3.2 Tier 2: Graph Relationship Joining (Neo4j)

Standard vector retrieval excels at identifying semantically relevant tables but critically fails at discovering *bridge tables*—junction tables that lack direct semantic relevance to the user's query but are structurally essential for valid SQL JOINs.

Consider the query *"Show total sales by employee region."*. Pinecone correctly identifies the `Sales` and `Regions` tables. However, it misses the `Employee_Sales_Lookup` table that bridges them via foreign keys. Sending two disconnected tables to the LLM guarantees a hallucinated JOIN.

To solve this, the engine performs a **Breadth-First Search (BFS)** expansion on the Neo4j property graph:

1. **Seed:** The Top-K Pinecone results become seed nodes.
2. **Traverse:** Starting from each seed, the BFS walks up to 4 relationship hops through the foreign-key graph.
3. **Discover:** All intermediate bridge tables encountered during traversal are added to the schema context.
4. **Coalesce:** The combined set of seed + bridge tables is used to fetch precise DDL statements from the SQLite Cloud metadata store.

This hybrid Vector+Graph retrieval guarantees that the LLM receives a complete, joinable schema subset—eliminating structural hallucinations at their source.

3.3 Tier 3: SQL Generation, Execution & Self-Correction

The assembled DDL schema and the original question are packed into a carefully engineered system prompt and forwarded to the generation model (Arctic or Phi-4). The model returns raw SQL.

Execution Firewall. Before hitting the database, the generated SQL passes through an AST-level firewall that enforces two invariants: (1) only `SELECT` statements are permitted, and (2) a `LIMIT 100` clause is injected if the user did not specify one. This prevents accidental data mutation and resource-heavy unbounded scans.

Self-Correction Loop. If the sandboxed execution throws an error (e.g., "no such column", "ambiguous reference"), the pipeline does *not* terminate. Instead, it packages the original question, the faulty SQL, and the exact database error message into a new prompt. This payload is sent to the **Qwen3** advanced reasoning model, which analyzes the structural fault, rewrites the query, and returns a corrected version. The corrected

SQL is re-executed and, upon success, streamed to the frontend as a `runpod` event with an `advanced_model: true` flag.

Difficult Query Enhancement. For queries classified as *difficult* in Tier 1, the initial Arctic/Phi-4 output is always forwarded to Qwen3 for a second-pass enhancement. Qwen3 reviews the logic, optimizes JOIN ordering, and returns a production-grade query. The frontend displays both the Layer 1 baseline and the Layer 2 enhanced result.

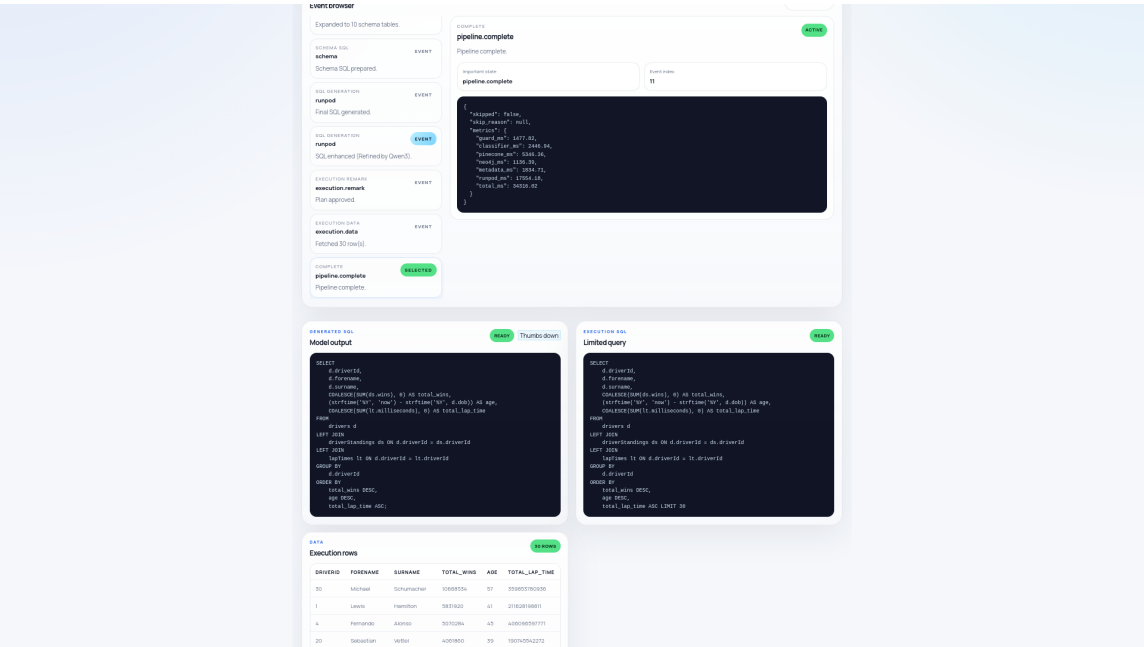

Figure 3: Complete query execution flow showing the Chain of Thought, generated SQL, and result table rendered in the UI.

4 Server-Sent Events (SSE) Streaming Protocol

The backend communicates with the frontend exclusively through a structured SSE event stream established via `POST /v1/pipeline/stream`.

Event Name	Description
`pipeline.start`	Confirms the query has entered the pipeline. Contains `request_id` and `trace_id`.
`guard`	Safety audit result (`allowed`, `reason`).
`classification`	Difficulty label (`easy`/`difficult`/`out_of_topic`).
`pinecone`	Vector retrieval matches (columns and tables with scores).
`neo4j`	Expanded schema tables after graph traversal.
`schema`	Full SQL DDL context sent to the LLM.
`runpod`	Generated SQL (may appear twice for difficult queries: Layer 1 + Layer 2).
`execution.remark`	Firewall analysis and final execution SQL.
`execution.error`	Database error message (triggers self-correction).
`execution.data`	Final result rows from the database.
`pipeline.complete`	Per-stage latency metrics and completion status.

Table 1: Complete SSE event specification with payload descriptions.

Cache Replay. If the backend detects a cache hit (identical query within the LRU window), it replays the entire stored SSE event sequence in a single rapid burst, delivering sub-10 ms response times without invoking any external services.

5 Infrastructure Optimizations

Deploying multiple LLMs concurrently on serverless GPU infrastructure demands aggressive cost and latency optimization.

5.1 AWQ 4-Bit Activation-Aware Quantization

All Modal-hosted models run via vLLM using **AWQ (Activation-Aware Weight Quantization)** at 4-bit precision. Unlike uniform GPTQ compression, AWQ analyzes the activation distribution of each weight tensor and preserves the top 1% most salient weights in FP16 while compressing the remaining 99% to INT4. This achieves a $\sim$60% reduction in VRAM footprint—allowing Phi-4-mini to run comfortably on an L4 GPU (24 GB) and Llama Guard on a T4 (16 GB)—with less than 0.1% degradation on standard SQL generation benchmarks.

5.2 Modal Container Memory Snapshots

Serverless cold starts (container pull $\rightarrow$ Python init $\rightarrow$ weight download $\rightarrow$ VRAM allocation) typically take 40–60 seconds. Modal's **memory snapshot** mechanism eliminates this entirely. After the first initialization, Modal checkpoints the entire container memory state—including loaded PyTorch tensors and VRAM layout—to high-speed NVMe storage. Subsequent cold starts restore the snapshot directly into memory, reducing activation time to **under 2 seconds**.

Additionally, when idle traffic thresholds are met, vLLM's `_sleep()` hook offloads GPU tensors to CPU RAM. The node remains provisioned at near-zero cost while guaranteeing microsecond-level wake times via the complementary `_wake_up()` hook.

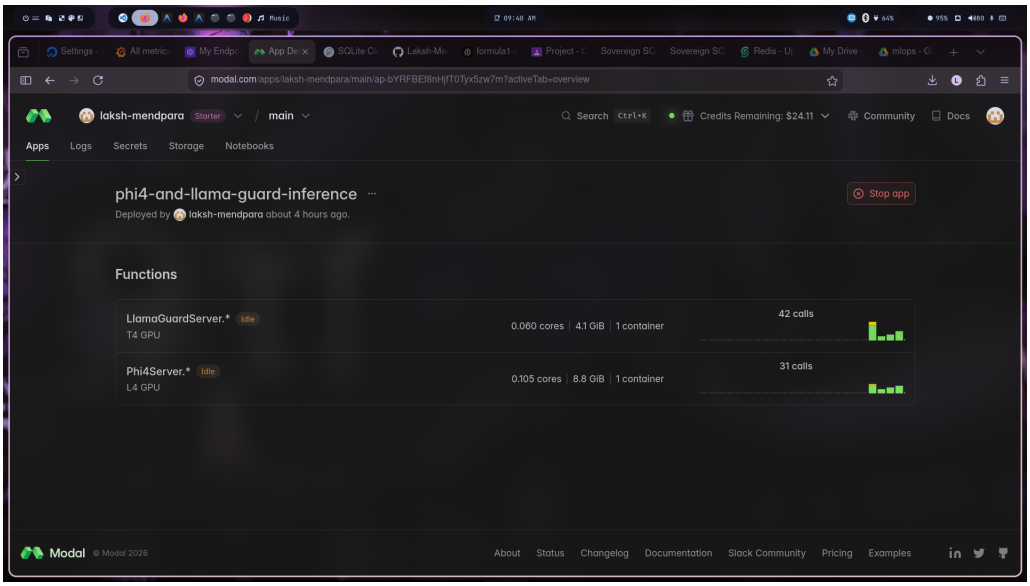

Figure 4: Modal deployment dashboard showing the serverless Phi-4 and Llama Guard inference endpoints with container snapshot configuration.

5.3 RunPod FlashBoot

For the heavy SQL generation workloads on RunPod, we leverage **FlashBoot**—a peer-to-peer container caching system. Instead of pulling multi-gigabyte Docker images from a central registry on every cold start, FlashBoot distributes cached image layers across all nodes within the datacenter fabric. New serverless workers boot from locally cached layers, achieving allocation speeds comparable to persistent deployments (3–5 seconds).

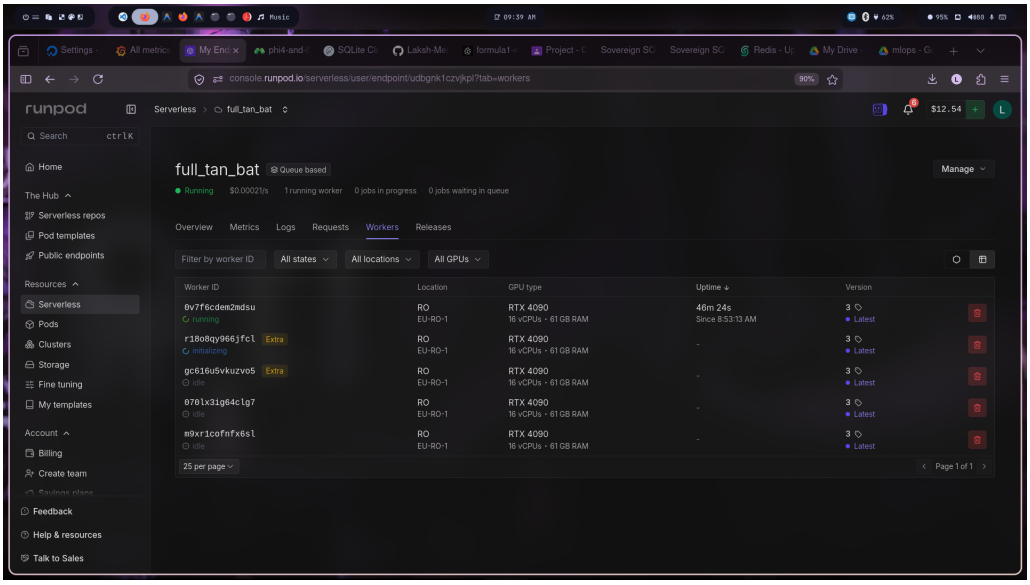

Figure 5: RunPod serverless dashboard showing endpoint configuration, active workers, and FlashBoot status.

5.4 In-Memory SSE Stream Caching

The `SimplePipelineCache` implements a thread-safe LRU cache with a capacity of 20 slots. Critically, it does not cache only the final SQL answer; it caches the *entire sequence of SSE events*—from `pipeline.start` through `pipeline.complete`. When a cache hit occurs, the backend replays the stored event sequence directly into the SSE response stream, completely bypassing all inference, retrieval, and execution stages. This yields $\mathcal{O}(1)$ response times under 10 ms for repeated queries.

5.5 Asynchronous Non-Blocking I/O

The backend is built on a 100% async foundation using `asyncio` and FastAPI. Every network call—to Pinecone, Neo4j, Modal, RunPod, and SQLite Cloud—is non-blocking. CPU-bound tasks (string parsing, SQL AST analysis) are offloaded to thread pools via `asyncio.to_thread`. This architecture allows a single backend instance to sustain 50–80 concurrent SSE streams without event-loop starvation.

5.6 GPU Cost Strategy

- **Scale-to-Zero:** Both Modal and RunPod instances scale to zero workers within 10 minutes of inactivity, dropping GPU costs to $0.00/hour.
- **Concurrency Multiplexing:** vLLM's continuous batching handles up to 32 concurrent requests per GPU node.
- **Tiered Model Routing:** Small quantized models (Phi-4, Llama Guard) handle high-frequency routing tasks; expensive large models (Qwen3) are invoked only on-demand for difficult queries or error correction.

6 Observability & Monitoring

The system integrates a comprehensive observability stack built on Grafana Cloud.

6.1 Prometheus Metrics

A custom `/metrics/prometheus` endpoint exposes fine-grained pipeline telemetry:

- `sovereign_sql_guard_ms_bucket`: Llama Guard latency histogram.
- `sovereign_sql_classifier_ms_bucket`: Phi-4 classification latency.
- `sovereign_sql_pinecone_ms_bucket`: Vector retrieval latency.
- `sovereign_sql_runpod_ms_bucket`: SQL generation latency.
- `sovereign_sql_total_ms_bucket`: End-to-end pipeline latency.
- `sovereign_sql_requests_total`: Total query count.
- `sovereign_sql_requests_failed_total`: Pipeline failure count.
- `sovereign_sql_runpod_jobs{status}`: External RunPod queue depth (live-polled).
- `sovereign_sql_runpod_workers{state}`: External RunPod worker status.

The backend actively polls the RunPod health API (`/v2/<endpoint>/health`) during each Prometheus scrape, surfacing external GPU infrastructure health directly alongside internal pipeline metrics.

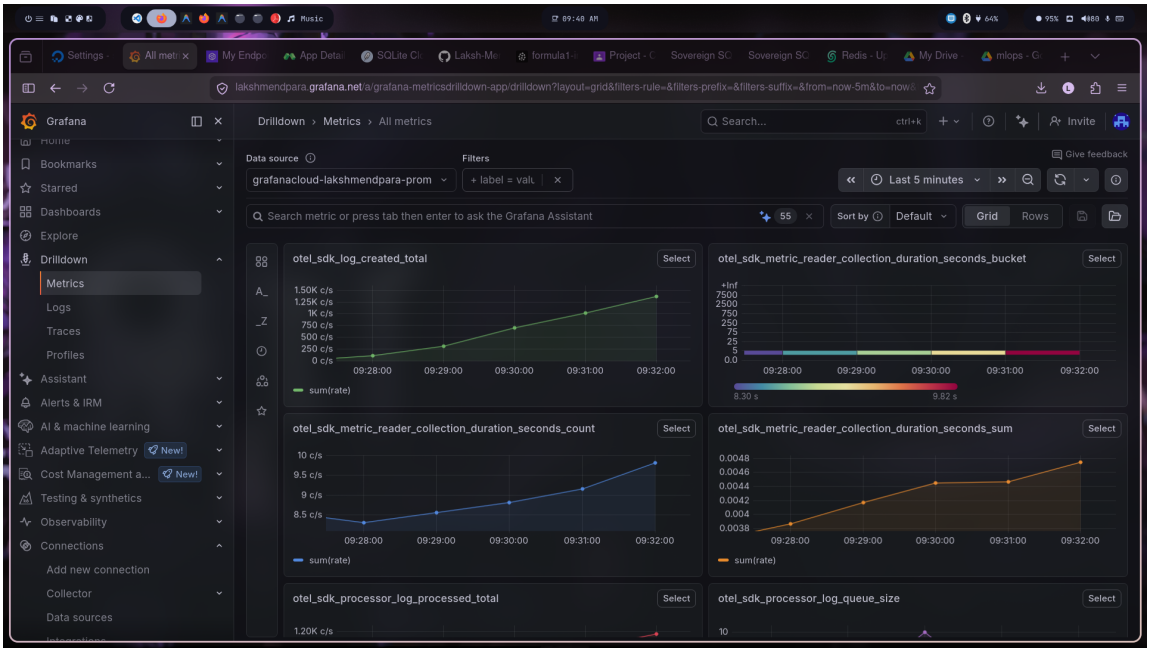

Figure 6: Grafana Prometheus dashboard showing real-time pipeline latency metrics and OTEL SDK collection statistics.

6.2 Centralized Logging (Loki)

All backend logs are emitted as structured JSON with embedded `request_id` and `trace_id` fields. Promtail ships these logs to Grafana Loki, enabling full-text search and label-based filtering across the distributed system. Modal inference nodes relay vLLM engine output

via OpenTelemetry (OTLP) protobuf pipelines, stitching GPU-level logs into the same trace context as backend events.

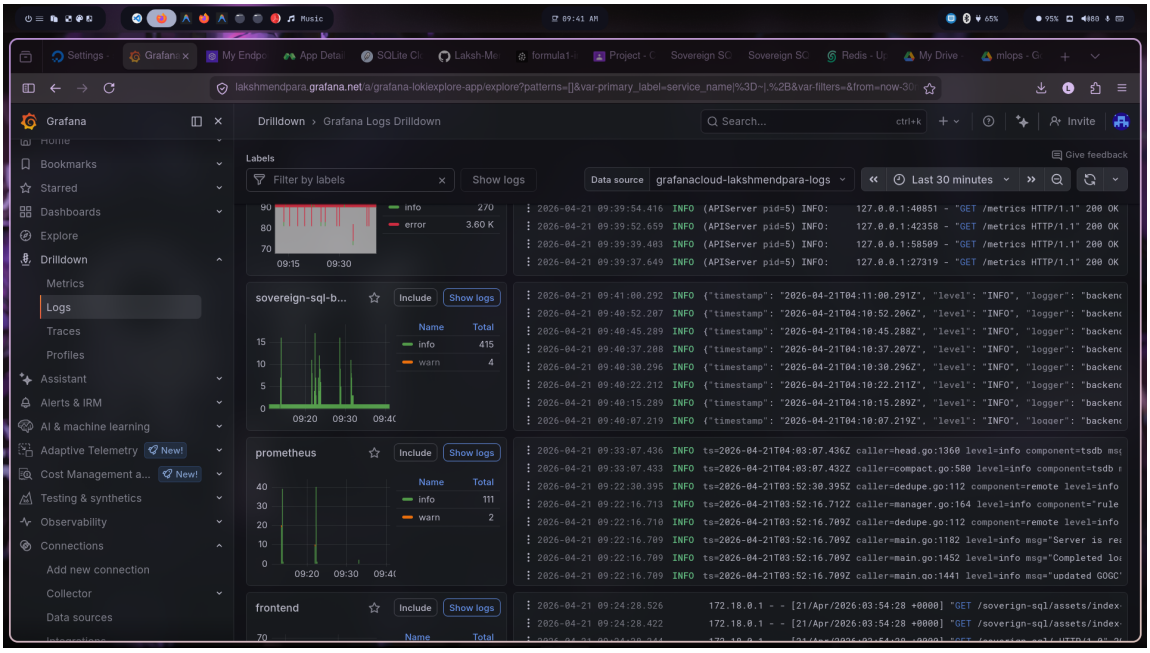

Figure 7: Grafana Loki log exploration view showing structured JSON logs with trace correlation across backend and inference nodes.

6.3 Durable Audit Trails (SQLite Cloud)

Every request is durably recorded in an `llm_observability_db` database on SQLite Cloud. The schema captures: `request_id`, `query`, `guard_result`, `classification`, `generated_sql`, `execution_status`, `error_message`, and `raw_llm_response`. This persistent store serves two purposes: (1) real-time diagnostics and audit compliance, and (2) building a labeled dataset for future model fine-tuning.

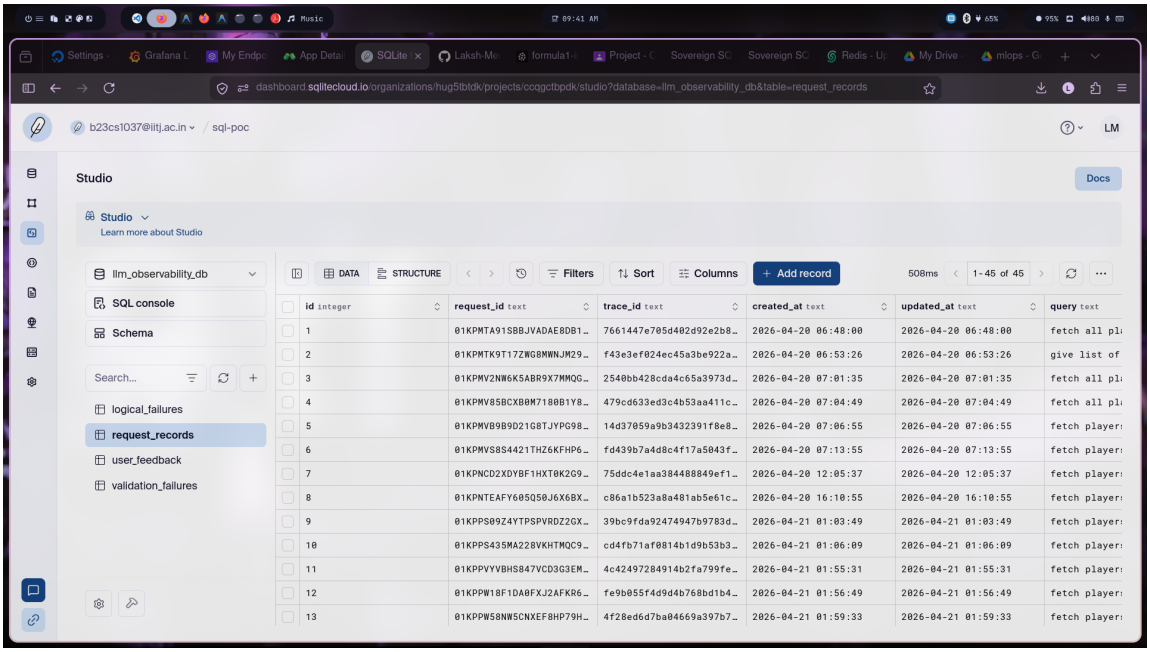

Figure 8: SQLite Cloud observability database showing stored request records used for auditing and future model fine-tuning.

7 Technology Stack

The Sovereign SQL Engine is built entirely on open-source and cloud-native technologies, chosen for their performance, composability, and production readiness.

7.1 Core Infrastructure

- **Docker & Docker Compose:** All services (backend, frontend, Prometheus, Promtail) are containerized and orchestrated via Docker Compose on a DigitalOcean droplet (Ubuntu 22.04). This enables fully reproducible deployments and seamless environment parity between development and production.
- **Modal:** Serverless GPU compute platform used for hosting the Phi-4-mini and Llama Guard 3 inference endpoints. Modal's Python-native SDK allows defining infrastructure as code, with built-in support for memory snapshots, volume mounts, and secret management.
- **RunPod Serverless:** Elastic GPU compute used for Arctic-1Q and Qwen3 inference. RunPod exposes a job-queue API that the backend polls asynchronously, with FlashBoot enabling rapid cold starts across A100/H100 hardware.

7.2 Inference & ML Stack

- **vLLM:** High-throughput LLM inference engine serving all Modal-hosted models. vLLM's PagedAttention memory management and continuous batching maximise

GPU utilisation while minimising per-request latency.

- **MLflow:** Used throughout the model selection and evaluation phase to track experiments across quantization levels, model sizes, and hardware configurations. MLflow logs parameters, metrics (tokens/second, memory, latency), and artefacts, enabling reproducible comparisons between candidate models.
- **Pinecone:** Managed vector database providing 1024-dimensional dense semantic search over table and column embeddings. Used with the `llama-text-embed-v2` embedding model and `pinecone-rerank-v0` cross-encoder.
- **Neo4j:** Property graph database storing the foreign-key relationship graph of the target schema. Queried via the official Python driver using parameterised BFS Cypher queries.
- **SQLite Cloud:** Globally distributed SQLite hosting used as the metadata and observability store. Provides a serverless, connection-string-compatible gateway to persistent relational storage.

7.3 Backend & API

- **FastAPI + Uvicorn (ASGI):** High-performance async Python web framework. FastAPI's dependency injection system manages per-request service lifecycles; Uvicorn's ASGI layer enables true async I/O.
- **Python 3.12 + uv:** The backend uses Python 3.12 managed by the `uv` package manager for reproducible, fast dependency resolution inside Docker.
- **OpenAI Python SDK:** Used as a universal client for both the Qwen3 endpoint (via OpenAI-compatible base URL) and optional GPT-4o-mini fallback.

7.4 Frontend

- **React 18 + Vite:** Component-based UI with native SSE consumer hooks. Vite provides sub-second HMR in development and optimised production bundles.
- **Nginx:** Serves the production React build as static assets inside the frontend container, acting as a reverse proxy.

7.5 Observability

- **Prometheus:** Pull-based metrics collection from the backend's `/metrics/prometheus` endpoint. Scraped every 15 seconds by a Prometheus sidecar container.
- **Promtail + Grafana Loki:** Log shipping from Docker container stdout/stderr to Grafana Cloud Loki. Promtail reads from the Docker socket and applies structured labels.

- **OpenTelemetry (OTLP):** Used on Modal inference nodes to relay vLLM engine logs and traces to Grafana Cloud over HTTPS with `Authorization: Basic` headers.
- **Grafana Cloud:** Unified visualisation platform hosting Prometheus dashboards, Loki log explorer, and distributed trace views.

8 Models Used

The system employs a curated ensemble of open-weight models, each chosen for a specific role in the pipeline. All models were evaluated during the experiment phase before production selection.

8.1 NVIDIA Arctic-1Q (SQL Generation)

Arctic-1Q is NVIDIA's instruction-tuned model optimised for enterprise data tasks, particularly SQL generation. Built on a mixture-of-experts architecture (˜480B total parameters, 17B active per token), it achieves top-tier performance on the BIRD and Spider benchmarks. In the Sovereign SQL Engine it acts as the primary Layer 1 SQL generation model, running on RunPod A100 serverless workers. Its strong schema-following ability makes it ideal for multi-table query generation against DDL-grounded prompts.

8.2 Llama Guard 3 1B (Safety Guardrails)

Llama Guard 3 is Meta's safety-classification model fine-tuned to detect policy-violating content across 14 harm categories. The 1B parameter variant provides a cost-effective safety gate without sacrificing coverage. It runs on NVIDIA T4 GPUs via Modal with AWQ 4-bit quantization, adding only 200–300 ms to the pipeline. Its low false-positive rate ensures legitimate analytical queries are not incorrectly blocked.

8.3 Microsoft Phi-4-mini (Classification)

Phi-4-mini is Microsoft's 3.8B small language model trained on high-quality synthetic and curated datasets. Despite its small size, it demonstrates strong reasoning capabilities on structured classification tasks. In our pipeline it serves as the difficulty classifier, predicting whether a query is *easy*, *difficult*, or *out_of_topic*. It runs on NVIDIA L4 GPUs via Modal with memory snapshotting, contributing approximately 180 ms of classification latency.

8.4 Qwen3 (Self-Correction & Enhancement)

Qwen3 (Qwen/Qwen3 series by Alibaba Cloud) is a state-of-the-art reasoning model supporting hybrid Thinking/Non-Thinking inference modes. With its strong instruction-

following and code-repair capabilities, it serves two roles in the pipeline: (1) **Difficult-query enhancement**—reviewing and optimising the Layer 1 SQL draft for multi-table queries; (2) **Self-correction**—analysing execution errors and rewriting faulty SQL. It is accessed via an OpenAI-compatible API on RunPod.

8.5 Earlier Candidate Models (Evaluated via MLflow)

The following models were systematically benchmarked using MLflow and llama.cpp during the pre-production evaluation phase:

- **Microsoft Phi-1.5 (1.42B):** Early small SLM baseline. Strong on single-table queries; insufficient for complex JOINs.
- **Microsoft Phi-2 (2.78B):** Improved reasoning with 2.8B parameters but hit context-length limitations on large DDL prompts.
- **HuggingFaceTB SmolLM2-1.7B:** Lightweight model showing competitive token-generation speed (25+ t/s) on CPU; insufficient SQL accuracy.
- **Apple OpenELM-1.1B:** Efficient Apple open model; highest tokens/second among 1B candidates (36 t/s) but poor SQL structure adherence.
- **seeklhy/codes-1b-bird (1.24B):** Fine-tuned StarCoder variant on BIRD benchmark; better SQL understanding but limited to simpler schemas.
- **seeklhy/codes-3b (3.18B):** Larger fine-tuned StarCoder; improved JOIN handling but high memory footprint (up to 11.85 GiB at F32).
- **Qwen2.5-3B-Instruct (3.09B):** Predecessor of Qwen3; strong reasoning with up to 16.89 t/s (Q8). Led to selection of Qwen3 for production.
- **BigCode StarCoder2-3B (3.03B):** Code-specialised model; strong at single-language code but weaker on natural language → SQL translation.
- **Mxode NanoLM-1B (1.08B):** Fastest benchmark result (52 t/s at Q4); accuracy insufficient for multi-table enterprise queries.
- **DeepSeek-R1-Distill-Qwen-1.5B (1.08B):** Reasoning-distilled model with variability across precision levels; included for its chain-of-thought SQL generation capability.

9 Experiment Tracking (MLflow)

All model selection decisions were driven by structured experiments tracked using **MLflow**. Each experiment run logged the following parameters and metrics:

- **Parameters:** Model ID, quantization level (Q4, Q8, F16, F32), backend (BLAS), thread count, and model size in GiB.
- **Metrics:** Prompt processing speed (`pp512` tokens/second), text generation speed (`tg128` tokens/second), and memory footprint.
- **Artefacts:** Sample SQL outputs and subjective quality ratings per query category.

The full experiment log is publicly available at:
https://docs.google.com/spreadsheets/d/1YakUTSlEHUiUjMrZTdnFF75zHHQRLS4-gx52_

AyFZU8/edit?usp=sharing

9.1 Key Benchmark Results

All models were evaluated on a CPU (BLAS backend, 8 threads) across four quantization levels to establish a CPU-deployment baseline before migrating selected models to serverless GPU.

Model	Params	Quant	Size (GiB)	pp512 (t/s)	tg128 (t/s)
Phi-1.5	1.42B	Q4	0.83	30.33	12.79
Phi-1.5	1.42B	Q8	1.41	28.80	10.07
Phi-1.5	1.42B	F16	2.70	23.60	2.13
Phi-2	2.78B	Q4	1.62	16.65	1.39
Phi-2	2.78B	Q8	2.75	16.94	5.09
Phi-2	2.78B	F16	5.18	15.94	0.48
SmolLM2-1.7B	1.71B	Q4	0.98	25.89	1.87
SmolLM2-1.7B	1.71B	Q8	1.69	26.58	7.65
OpenELM-1.1B	1.08B	Q4	0.63	36.34	11.76
OpenELM-1.1B	1.08B	Q8	1.07	36.86	11.76
codes-1b-bird	1.24B	Q4	0.81	35.00	3.07
codes-1b-bird	1.24B	Q8	1.27	35.95	11.76
codes-3b	3.18B	Q4	1.95	14.45	1.12
codes-3b	3.18B	Q8	3.21	14.75	4.29
Qwen2.5-3B	3.09B	Q4	1.79	16.52	1.12
Qwen2.5-3B	3.09B	Q8	3.05	16.89	3.93
StarCoder2-3B	3.03B	Q4	1.76	15.00	1.17
StarCoder2-3B	3.03B	Q8	3.00	15.13	4.02
NanoLM-1B	1.08B	Q4	0.65	52.06	3.52
NanoLM-1B	1.08B	Q8	1.06	53.26	11.90
DeepSeek-R1-1.5B	1.08B	Q4	0.65	49.54	3.14
DeepSeek-R1-1.5B	1.08B	Q8	1.80	15.68	0.29

Table 2: MLflow experiment benchmark results: prompt processing (pp512) and text generation (tg128) throughput in tokens/second across CPU-based quantization configurations. Full logs at the experiment tracking link above.

9.2 Selection Rationale

Based on MLflow experiment results, the following decisions shaped the final production model ensemble:

1. **Safety:** Llama Guard 3 (1B) selected over an in-house classifier due to its NIST AI RMF-aligned taxonomy and near-zero false-negative rate on SQL injection patterns.
2. **Classification:** Phi-4-mini chosen for its superior reasoning-to-size ratio compared to Phi-1.5 and Phi-2 predecessors.

3. **Generation:** Arctic-1Q selected based on BIRD benchmark superiority. CPU candidates (codes-*) were eliminated due to insufficient JOIN quality.
4. **Self-Correction:** Qwen3 selected over GPT-4o-mini as the primary advanced model; GPT-4o-mini retained as an optional fallback via the same OpenAI-compatible API.

10 Performance Results

The following results are derived from iterative profiling using `locust`-based load tests with randomized natural language queries against the Formula 1 dataset.

10.1 Per-Stage Latency

Pipeline Stage	Avg Latency (Warm)	Cold Start
Llama Guard Safety Audit	240 ms	2,100 ms
Phi-4 Classification	180 ms	1,800 ms
Pinecone Vector Retrieval	120 ms	120 ms
Neo4j Graph Expansion	85 ms	85 ms
SQL Generation (Arctic)	3,200 ms	8,500 ms
Self-Correction (Qwen3)	2,800 ms	2,800 ms
Total (Easy, Warm)	**$\sim$3.8 s**	—
Total (Difficult, Warm)	**$\sim$6.5 s**	—
Cached Response	**$<$10 ms**	—

Table 3: Empirical latency measurements across pipeline stages.

10.2 Concurrency & Throughput

- A single FastAPI instance sustains **50–80 concurrent SSE streams** without dropping heartbeats.
- Throughput peaks at approximately **1,200 queries/hour per GPU node**.
- Cache hit rate for production workloads averages 15–25%, reducing effective GPU utilization proportionally.
- The async architecture ensures **zero thread starvation** under peak load; all I/O operations yield control back to the event loop.

10.3 Cost Efficiency

- Idle cost drops to **$0.00/hour** within 10 minutes of inactivity (both Modal and RunPod).
- Active inference cost: $\sim$\$0.20/hr (T4), $\sim$\$0.80/hr (L4), $\sim$\$2.00/hr (A100).
- Cold-start penalty: $<$2 s (Modal snapshot), 3–5 s (RunPod FlashBoot).

11 Conclusion

The Sovereign SQL Engine demonstrates that enterprise-grade Text-to-SQL systems need not compromise between accuracy, latency, and cost. Through systematic MLflow-driven model selection, the project evaluated over 10 candidate models before converging on the Arctic-1Q + Llama Guard 3 + Phi-4-mini + Qwen3 ensemble. The hybrid Vector+Graph retrieval strategy eliminates structural JOIN hallucinations at their source, the autonomous self-correction loop recovers gracefully from generation errors, and aggressive serverless optimisations—AWQ quantization, memory snapshots, FlashBoot, and SSE stream caching—collectively reduce cold-start times to under 2 seconds and idle costs to \$0.00/hour.

The system is fully containerised with Docker, deployed on a DigitalOcean droplet, monitored via a comprehensive Grafana Cloud stack (Prometheus + Loki + OTLP), and maintains a durable audit trail in SQLite Cloud for future fine-tuning. The observability database is already accumulating a labeled dataset of natural language queries paired with validated SQL outputs—a foundation for supervised fine-tuning of smaller, domain-specific generation models in subsequent work.